

```

1  -- 1. Views
2
3  -- a. Review this example view
4  CREATE OR ALTER VIEW Lab001.OrderSummary AS
5  SELECT OrderID
6  , SUM(ol.Quantity * p.ProductPrice) TotalAmount
7  FROM Lab001.OrderLine ol
8  JOIN Lab001.product p ON ol.ProductID = p.ProductID
9  GROUP BY OrderID
10 GO
11
12 -- b. Create a view, "Lab001.OrderDetail" based on the user-friendly
13 -- order details query below
14 --TODO():
15 SELECT oh.OrderDate,
16 oh.OrderID,
17 cu.FirstName,
18 cu.LastName,
19 cu.AddressLine1,
20 cu.City,
21 cu.RegionAbbr,
22 cu.PostalCode,
23 p.ProductName,
24 ol.Quantity,
25 p.ProductPrice
26 FROM Lab001.customer cu, Lab001.OrderHeader oh, Lab001.OrderLine ol, Lab001.product p
27 WHERE cu.CustomerID=oh.CustomerID AND
28 oh.OrderID=ol.OrderID AND
29 ol.ProductID=p.ProductID
30 GO
31
32 CREATE OR ALTER VIEW Lab001.OrderDetail AS
33 SELECT
34     oh.OrderDate,
35     oh.OrderID,
36     cu.FirstName,
37     cu.LastName,
38     cu.AddressLine1,
39     cu.City,
40     cu.RegionAbbr,
41     cu.PostalCode,
42     p.ProductName,
43     ol.Quantity,
44     p.ProductPrice
45 FROM Lab001.Customer cu
46 JOIN Lab001.OrderHeader oh ON cu.CustomerID = oh.CustomerID
47 JOIN Lab001.OrderLine ol ON oh.OrderID = ol.OrderID
48 JOIN Lab001.Product p ON ol.ProductID = p.ProductID;
49 GO
50
51 -- c. Select all rows and columns from Lab001.OrderDetail and take a screenshot
52 -- TODO():
53
54 SELECT * FROM Lab001.OrderDetail;
55
56 -- 2. Triggers
57
58 -- a. Review the table below
59 DROP TABLE IF EXISTS Lab001.ProductPriceAudit
60 GO
61 CREATE TABLE Lab001.ProductPriceAudit
62 (
63     UpdateDate date,
64     OldValue money,
65     NewValue money
66 )
67 GO
68
69 -- b. Review the trigger below

```

```

70 CREATE OR ALTER TRIGGER Lab001.AuditPriceChangeTrigger
71     ON [Lab001].[product]
72     FOR UPDATE
73 AS
74 BEGIN
75     SET NOCOUNT ON;
76     DECLARE @oldValue AS MONEY;
77     DECLARE @newValue AS MONEY;
78     SELECT @oldValue = ProductPrice from deleted;
79     SELECT @newValue = ProductPrice from inserted;
80     INSERT INTO Lab001.ProductPriceAudit VALUES (GETDATE(), @oldValue, @newValue);
81 END
82 GO
83
84 -- c. Increase price for product by $10
85 UPDATE Lab001.product
86 SET ProductPrice = ProductPrice + 10.00
87 WHERE ProductID = 101
88 GO
89
90 -- d. Verify trigger has worked
91 SELECT *
92 FROM Lab001.ProductPriceAudit
93 GO
94
95 -- e. Update the Lab001.ProductPriceAudit adding price_change_percentage column
96 -- TODO():
97
98 ALTER TABLE Lab001.ProductPriceAudit
99 ADD price_change_percentage DECIMAL(5,2);
100
101 -- f. Update the Lab001.AuditPriceChangeTrigger trigger to add the price percentage
102 -- change
103 -- TODO():
104
105 CREATE OR ALTER TRIGGER Lab001.AuditPriceChangeTrigger
106     ON [Lab001].[product]
107     FOR UPDATE
108 AS
109 BEGIN
110     SET NOCOUNT ON;
111     DECLARE @oldValue AS MONEY;
112     DECLARE @newValue AS MONEY;
113     DECLARE @priceChange DECIMAL(5,2);
114
115     SELECT @oldValue = ProductPrice from deleted;
116     SELECT @newValue = ProductPrice from inserted;
117
118     SET @priceChange = (@newValue - @oldValue) / @oldValue * 100;
119
120     INSERT INTO Lab001.ProductPriceAudit (UpdateDate, OldValue, NewValue,
121     price_change_percentage)
122     VALUES (GETDATE(), @oldValue, @newValue, @priceChange);
123 END
124 GO
125
126 -- g. Increase price for product by 10%
127 UPDATE Lab001.product
128 SET ProductPrice = ProductPrice * 1.1 -- 10% price increase
129 WHERE ProductID = 101
130 GO
131
132 -- h. Verify the trigger has worked by selecting all rows and
133 -- columns from Lab001.ProductPriceAudit and take a screenshot
134 -- TODO():
135
136 SELECT * FROM Lab001.ProductPriceAudit;
137
138 -- 3. Functions

```

```

137
138 -- a. Review the function below
139 CREATE OR ALTER FUNCTION Lab001.CalculateDiscountedPrice(
140 @productID INT,
141 @percentDiscount NUMERIC(10,3))
142 RETURNS MONEY
143 AS
144 BEGIN
145 DECLARE @returnValue AS MONEY;
146 SELECT @returnValue = ProductPrice * @percentDiscount
147 FROM [Lab001].[product]
148 WHERE [Lab001].[product].ProductID = @productID
149 RETURN(@returnValue)
150 END
151 GO
152
153
154 -- b. Run the function (experiment with different parameters)
155 SELECT Lab001.CalculateDiscountedPrice(102, .50)
156 GO
157
158 -- c. Create a new function called Lab001.calculate_discounted_percent that takes
159 -- a product id and a new price and returns the discount percentage
160 -- For example, if ProductID 103 was discounted from $59 to $29.50 it would return 0.5
161 -- (or 50%)
162 CREATE OR ALTER FUNCTION Lab001.calculate_discounted_percent
163 (
164     @productID INT,
165     @newPrice MONEY
166 )
167 RETURNS DECIMAL(5, 2)
168 AS
169 BEGIN
170     DECLARE @originalPrice MONEY;
171
172     SELECT @originalPrice = ProductPrice
173     FROM Lab001.product
174     WHERE ProductID = @productID;
175
176     IF @originalPrice IS NULL
177     BEGIN
178         RETURN NULL;
179     END
180
181     RETURN CAST(((@originalPrice - @newPrice) / @originalPrice * 100) AS DECIMAL(5, 2));
182 END
183 GO
184
185 -- d. Run the function discounting ProductID 103 to $29.50 and take a screenshot
186 -- TODO():
187
188 SELECT Lab001.calculate_discounted_percent(103, 29.50);

```